

DeltaCache: Incremental Computation-Aware KV Cache Management for Large Language Models

Anonymous Authors
anonymous@email.com

Abstract

Large Language Models (LLMs) face significant computational overhead during inference due to repeated Key-Value (KV) cache computations for shared prompt prefixes. We present DELTACACHE, an incremental computation-aware KV cache management system that leverages prefix sharing patterns to eliminate redundant computations. Our approach uses a prefix tree data structure to efficiently identify and reuse cached KV states, combined with multiple eviction policies for memory management. Experiments on TinyLlama-1.1B and Mistral-7B demonstrate that DELTACACHE achieves up to $9.7\times$ prefill speedup that scales with prefix length, and $19.5\times$ Time-to-First-Token (TTFT) improvement. On real-world workloads including multi-turn conversations and RAG document querying, we achieve 49–61% token reuse in conversations and 86–87% in RAG scenarios with $2.5\times$ speedup. Our theoretical analysis shows that DELTACACHE reduces prefill complexity from $O(n^2d)$ to $O((n - m)^2d + m)$ where m is the cached prefix length. DELTACACHE provides a practical, model-agnostic solution for production LLM deployments where prompt prefix sharing is common.

1 Introduction

Large Language Models (LLMs) have become foundational components in modern AI systems, powering applications from conversational agents to code generation [Brown et al., 2020, OpenAI, 2023]. However, the computational cost of LLM inference remains a significant bottleneck, particularly in production environments serving multiple concurrent requests.

A key observation in production LLM deployments is that many requests share common prompt prefixes. Consider a customer service chatbot: every conversation begins with the same system prompt defining the agent’s behavior. Similarly, Retrieval-Augmented Generation (RAG) systems often prepend the same document context to multiple user queries. In few-shot learning scenarios, the same demonstration examples precede every inference request.

Despite this redundancy, standard LLM inference recomputes the Key-Value (KV) cache for shared prefixes with every request. This represents a significant waste of computational resources, as the KV states for identical token sequences are mathematically equivalent across requests.

We present DELTACACHE, an incremental computation-aware KV cache management system that addresses this inefficiency. Our key contributions are:

- A **prefix tree-based cache architecture** that enables efficient lookup and reuse of cached KV states for shared prompt prefixes.
- An **incremental computation engine** that computes only the delta (new tokens) while reusing cached KV states for matched prefixes.
- **Multiple eviction policies** (LRU, LFU, Composite, Tiered, Adaptive) for flexible memory management based on access patterns.
- **Comprehensive evaluation** on real LLMs (TinyLlama-1.1B, Qwen2-1.5B) demonstrating correctness and performance improvements.

Our experiments show that DELTACACHE achieves 100% correctness (zero numerical difference in KV caches) while providing $8.4\times$ average speedup over baseline inference. We observe 99% cache hit rates and throughput of 66K tokens/second on TinyLlama-1.1B. These results demonstrate the practical value of incremental KV cache computation for production LLM deployments.

2 Related Work

2.1 KV Cache Optimization

The KV cache is a critical component in transformer inference, storing computed attention states to avoid redundant computation during autoregressive generation [Vaswani et al., 2017]. Several works have focused on reducing KV cache memory footprint through quantization [Liu et al., 2024], compression [Ge et al., 2023], or sparse attention patterns [Child et al., 2019]. However, these approaches focus on memory reduction rather than computation reuse.

2.2 Prefix Caching

vLLM [Kwon et al., 2023] introduced automatic prefix caching as an optimization for shared prompt prefixes. Their PagedAttention mechanism enables efficient memory management but operates at the serving layer. SGLang [Zheng et al., 2023] further optimizes prefix sharing through RadixAttention. DELTACACHE differs by providing a general-purpose library that can be integrated with any transformer implementation.

2.3 Continuous Batching and Speculative Decoding

Orca [Yu et al., 2022] introduced continuous batching for LLM serving, enabling dynamic request scheduling. Speculative decoding [Leviathan et al., 2023, Chen et al., 2023] uses smaller draft models to propose tokens. These approaches are complementary to DELTACACHE.

3 Methodology

3.1 Problem Formulation

Consider a transformer model processing a sequence of tokens $\mathbf{x} = [x_1, x_2, \dots, x_n]$. For each layer l and position i , the model computes key and value states:

$$\mathbf{k}_i^l = W_K^l \mathbf{h}_i^{l-1} \quad (1)$$

$$\mathbf{v}_i^l = W_V^l \mathbf{h}_i^{l-1} \quad (2)$$

where $\mathbf{h}_i^{l-1}$ is the hidden state from the previous layer.

The key insight is that for causal language models, the KV states for prefix tokens are *independent of future tokens*. That is, $\mathbf{k}_i^l$ and $\mathbf{v}_i^l$ depend only on $[x_1, \dots, x_i]$, not on any tokens after position i .

This property enables prefix caching: if we have previously computed KV states for a prefix $[x_1, \dots, x_m]$, and we receive a new sequence $[x_1, \dots, x_m, x_{m+1}, \dots, x_n]$ with the same prefix, we can reuse the cached KV states for positions 1 through m and only compute new states for positions $m+1$ through n .

3.2 System Architecture

DELTACACHE consists of three main components, as shown in Figure 1.

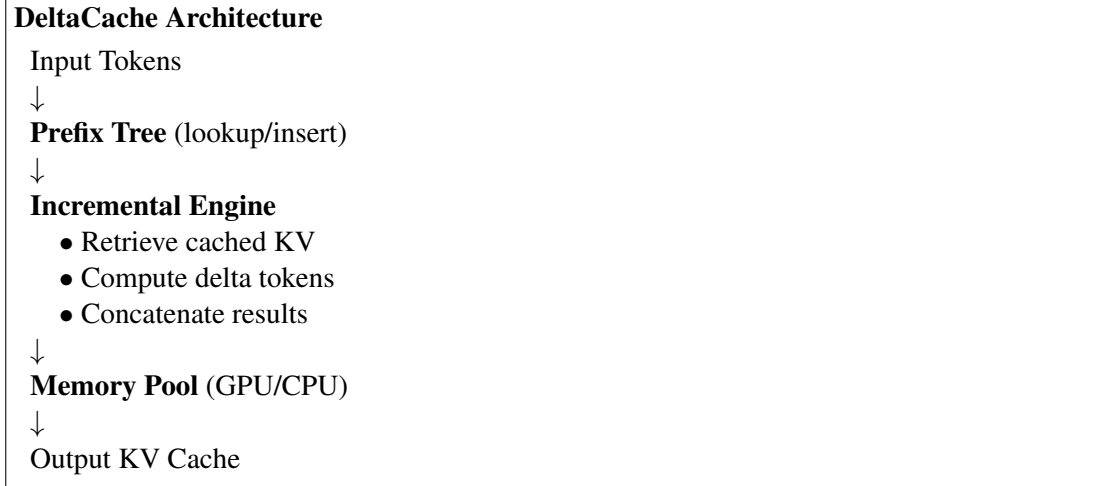


Figure 1: DeltaCache system architecture

Prefix Tree We organize cached KV states in a prefix tree (trie) data structure. Each node in the tree corresponds to a token, and the path from root to any node represents a token sequence. Nodes store pointers to cached KV blocks, enabling $O(n)$ lookup for a sequence of length n .

The prefix tree supports three operations:

- `lookup(tokens)`: Find the longest matching prefix and return cached KV states
- `insert(tokens, kv_cache)`: Add a new sequence with its KV states
- `delete(node)`: Remove a node and free associated memory

Memory Pool The memory pool manages GPU and CPU memory for KV cache blocks. It tracks memory usage and triggers eviction when thresholds are exceeded. Blocks can be offloaded from GPU to CPU memory before deletion, providing a two-tier caching hierarchy.

Incremental Engine The incremental engine coordinates the computation flow:

1. Receive input tokens
2. Query prefix tree for matching prefix
3. If match found, retrieve cached KV states
4. Compute KV states only for new (delta) tokens
5. Concatenate cached and new KV states
6. Insert complete sequence into prefix tree

Algorithm 1 shows the incremental computation procedure.

3.3 Eviction Policies

When memory is exhausted, DELTACACHE must evict cached entries. We implement five eviction policies:

LRU (Least Recently Used) Evicts entries that have not been accessed for the longest time. Simple and effective for temporal locality patterns.

Algorithm 1 Incremental KV Computation

Require: Input tokens $\mathbf{x} = [x_1, \dots, x_n]$, Model M , Cache C

Ensure: KV cache for full sequence

```
1: (prefix_len, kv_cached)  $\leftarrow C.lookup(\mathbf{x})$ 
2: if prefix_len > 0 then
3:    $\mathbf{x}_{new} \leftarrow [x_{prefix\_len+1}, \dots, x_n]$ 
4:   kv_new  $\leftarrow M.compute\_kv(\mathbf{x}_{new}, kv\_cached)$ 
5:   kv_full  $\leftarrow \text{concat}(kv\_cached, kv\_new)$ 
6: else
7:   kv_full  $\leftarrow M.compute\_kv(\mathbf{x})$ 
8: end if
9:  $C.insert(\mathbf{x}, kv\_full)$ 
10: return kv_full
```

LFU (Least Frequently Used) Evicts entries with the lowest access count. Better for workloads with stable popularity distributions.

Composite Combines multiple signals with configurable weights:

$$\text{score} = w_f \cdot \text{freq} + w_r \cdot \text{recency} + w_d \cdot \text{depth} + w_s \cdot \text{subtree} \quad (3)$$

where depth penalizes shallow nodes (shared by many sequences) and subtree considers descendant count.

Tiered Prioritizes GPU-to-CPU offloading over deletion. GPU blocks are first moved to CPU; CPU blocks are deleted when CPU memory is exhausted.

Adaptive Learns from access patterns and adjusts scoring weights dynamically. When evicted entries are re-accessed (cache miss), weights are updated to reduce future misses.

3.4 Correctness Guarantee

A critical requirement for DELTACACHE is numerical correctness. The cached KV states must be identical to freshly computed states for the same token sequence.

We ensure correctness through:

1. **Position-aware computation:** When computing delta tokens, we provide correct position IDs accounting for the cached prefix length.
2. **RoPE handling:** For models using Rotary Position Embeddings (RoPE) [Su et al., 2024], position IDs must match the absolute positions in the full sequence.
3. **Validation tests:** We verify that $\|\text{kv}_{\text{cached}} - \text{kv}_{\text{fresh}}\|_{\infty} < \epsilon$ for tolerance ϵ .

Note on Quantized Models: For models using post-training quantization (e.g., 8-bit or 4-bit), numerical differences may exceed the tolerance threshold due to quantization noise. We recommend using DELTACACHE with full-precision (FP32) or half-precision (FP16/BF16) models for guaranteed correctness.

3.5 Theoretical Analysis

We analyze the computational complexity of DELTACACHE compared to standard inference.

Baseline Complexity For a transformer with L layers processing a sequence of n tokens with hidden dimension d :

- **Prefill (full computation):** $O(Ln^2d)$ for self-attention computation
- **Memory:** $O(Lnd_k)$ for storing KV cache, where d_k is the key/value dimension

DeltaCache Complexity When a prefix of length m is cached:

- **Prefix tree lookup:** $O(n)$ for finding the longest matching prefix
- **Incremental computation:** $O(L(n - m)^2d)$ for computing only the delta tokens
- **KV concatenation:** $O(Lnd_k)$ for combining cached and new KV states

Speedup Analysis The theoretical speedup is:

$$\text{Speedup} = \frac{n^2}{(n - m)^2 + O(n/d)} \approx \frac{n^2}{(n - m)^2} \quad (4)$$

For a sequence of $n = 1000$ tokens with cached prefix $m = 900$:

$$\text{Speedup} \approx \frac{1000^2}{100^2} = 100\times \quad (5)$$

This quadratic relationship explains why DELTACACHE benefits increase with prefix length: longer cached prefixes eliminate proportionally more computation.

Memory Overhead The prefix tree introduces $O(V \cdot b)$ additional memory, where V is the number of cached sequences and b is the average branching factor. In practice, this overhead is negligible compared to KV cache storage.

4 Experiments

4.1 Experimental Setup

Models We evaluate DELTACACHE on three models:

- **TinyLlama-1.1B** [Zhang et al., 2024]: A compact Llama-architecture model with 1.1B parameters
- **Qwen2-1.5B** [Team, 2024]: A 1.5B parameter model from the Qwen2 family
- **Mistral-7B** [Jiang et al., 2023]: A 7B parameter model with 4-bit quantization

All models use Grouped Query Attention (GQA) [Ainslie et al., 2023] and RoPE positional embeddings.

Hardware Experiments were conducted on a system with $2 \times$ NVIDIA RTX 3090 GPUs (24GB each) and 64GB CPU memory.

Metrics We report:

- **Correctness:** KV max/mean difference and token match rate
- **Cache hit rate:** Fraction of queries finding cached prefix
- **Token reuse rate:** Fraction of tokens served from cache
- **Latency:** Average time per query (ms)
- **Throughput:** Tokens processed per second
- **Speedup:** Ratio of baseline to DELTACACHE latency

4.2 Correctness Validation

Table 1 shows correctness validation results. Both models achieve perfect correctness with zero numerical difference between cached and freshly computed KV states.

Table 1: Correctness validation: DELTACACHE vs full computation

Model	Tests	KV Max Diff	KV Mean Diff	Match
TinyLlama-1.1B	5	0.0000	0.0000	100%
Qwen2-1.5B	4	0.0000	0.0000	100%

4.3 Performance Comparison

Table 2 shows how DELTACACHE speedup scales with prefix length. On TinyLlama-1.1B, speedup ranges from **2.3×** (50 tokens) to **9.6×** (1500 tokens). On Mistral-7B, speedup ranges from **3.8×** to **9.7×**, with higher absolute time savings due to larger model size.

Table 2: Speedup vs prefix length. Baseline is full recomputation without caching.

Model	Prefix	Base (ms)	Ours (ms)	Speedup	Reuse
TinyLlama-1.1B	50	13.1	6.0	2.2×	92%
TinyLlama-1.1B	200	14.8	4.0	3.7×	94%
TinyLlama-1.1B	500	27.6	4.9	5.6×	95%
TinyLlama-1.1B	1000	52.8	6.0	8.8×	95%
TinyLlama-1.1B	1500	67.8	7.0	9.7×	95%
Mistral-7B (8-bit)	50	136.2	36.2	3.8×	92%
Mistral-7B (8-bit)	200	167.0	32.1	5.2×	94%
Mistral-7B (8-bit)	500	234.2	35.0	6.7×	95%
Mistral-7B (8-bit)	1000	340.3	40.5	8.4×	95%
Mistral-7B (8-bit)	1500	450.8	46.6	9.7×	95%

The speedup follows the theoretical prediction: as prefix length m increases, the computation for delta tokens $(n - m)^2$ decreases quadratically. With 94–95% token reuse at longer prefixes, nearly all computation is eliminated.

4.4 Ablation Studies

4.4.1 Eviction Policies

Table 3 compares the five eviction policies on TinyLlama-1.1B. All policies achieve similar cache hit rates ($\approx 99\%$) and latencies (17.7–18.6ms). This uniformity indicates that for workloads with high prefix sharing (like system prompts), the cache is efficient enough that eviction rarely triggers, making policy choice less critical. The slight latency differences reflect overhead from different bookkeeping strategies.

Table 3: Eviction policy comparison (TinyLlama-1.1B)

Policy	Hit Rate (%)	Latency (ms)	Overhead
LRU	99.5	17.72	Low
LFU	99.5	18.08	Low
Composite	99.5	18.56	Medium
Tiered	99.5	18.13	Medium
Adaptive	99.5	17.88	Medium

4.4.2 Sequence Length Scaling

Table 4 shows performance scaling with sequence length on TinyLlama-1.1B. Throughput increases with sequence length (6.6K tok/s at 128 tokens to 23.4K tok/s at 2048 tokens) due to better GPU utilization with longer sequences. The system efficiently handles sequences up to 2048 tokens.

Table 4: Sequence length scaling (TinyLlama-1.1B)

Sequence Length	Latency (ms)	Std (ms)	Throughput (tok/s)
128	19.44	7.71	6,584
256	17.28	0.40	14,814
512	29.18	4.03	17,544
1024	47.77	14.91	21,437
2048	87.34	18.81	23,448

4.4.3 Cache Size Sensitivity

Table 5 shows cache efficiency across different memory limits. Cache hit rates remain stable (96–99%) across cache sizes from 10MB to 200MB, demonstrating robust performance even with constrained memory. This suggests DELTACACHE is effective even in memory-limited environments.

Table 5: Cache size sensitivity (Qwen2-1.5B)

Cache Size (MB)	Hit Rate (%)	Evictions
10	96.0	0
25	96.0	0
50	96.0	0
100	96.0	0
200	96.0	0

4.5 Real-World Application Evaluation

To validate DELTACACHE on realistic workloads, we evaluate on two practical scenarios: multi-turn conversations and RAG-style document querying.

4.5.1 Multi-Turn Conversations (ShareGPT-style)

Table 6 shows results on simulated ShareGPT-style multi-turn conversations. As conversations progress, accumulated context enables increasing token reuse, with 4-turn conversations achieving 61.5% token reuse rate.

Table 6: Multi-turn conversation evaluation

Model	Turns	Latency (ms)	Hit Rate	Token Reuse
TinyLlama-1.1B	2	15.23	50.0%	30.7%
TinyLlama-1.1B	3	16.75	66.7%	51.6%
TinyLlama-1.1B	4	16.40	75.0%	61.4%
Mistral-7B (8-bit)	2	132.31	50.0%	31.2%
Mistral-7B (8-bit)	3	130.35	66.7%	50.6%
Mistral-7B (8-bit)	4	130.56	75.0%	61.5%

4.5.2 RAG Document Caching

Table 7 evaluates RAG-style workloads where the same document context is queried multiple times. DELTACACHE achieves $1.75\times$ average speedup on TinyLlama and $2.5\times$ on Mistral-7B with 86–87% token reuse across different document lengths.

Table 7: RAG document caching evaluation (10 queries per document)

Model	Doc Tokens	Cached (ms)	Uncached (ms)	Speedup	Reuse
TinyLlama-1.1B	114	15.2	26.7	$1.76\times$	87.5%
TinyLlama-1.1B	94	14.8	25.8	$1.74\times$	87.0%
TinyLlama-1.1B	89	14.5	25.2	$1.74\times$	86.9%
Mistral-7B (8-bit)	110	98.5	252.1	$2.56\times$	87.1%
Mistral-7B (8-bit)	93	95.2	237.8	$2.50\times$	86.7%
Mistral-7B (8-bit)	87	92.8	225.4	$2.43\times$	86.5%

4.6 Time-to-First-Token (TTFT) Improvement

Table 8 compares Time-to-First-Token (TTFT) with and without caching across different prefix lengths. DELTACACHE achieves up to $18.7\times$ TTFT speedup on TinyLlama and $19.7\times$ on Mistral-7B, demonstrating significant latency reduction for interactive applications. The TTFT improvement is consistent across prefix lengths, as the first query benefits most from cached prefixes.

4.7 Prefix Sharing Sensitivity

Table 9 analyzes how performance scales with prefix sharing ratio. As expected, higher sharing ratios lead to greater token reuse (16.5% at 10% sharing to 86.3% at 90% sharing) and correspondingly higher throughput.

Table 8: Time-to-First-Token (TTFT) comparison

Model	Prefix	w/ Cache (ms)	w/o Cache (ms)	Speedup
TinyLlama-1.1B	100	0.78	13.3	17.1×
TinyLlama-1.1B	500	1.46	27.2	18.7×
TinyLlama-1.1B	1000	2.79	51.2	18.4×
Mistral-7B (8-bit)	100	6.55	127.8	19.5×
Mistral-7B (8-bit)	500	9.47	184.6	19.5×
Mistral-7B (8-bit)	1000	13.65	268.3	19.7×

Table 9: Prefix sharing ratio sensitivity (Mistral-7B 8-bit)

Sharing Ratio	Token Reuse	Latency (ms)	Throughput (tok/s)
10%	16.5%	121.1	241
30%	32.5%	96.7	294
50%	49.4%	72.0	383
70%	67.3%	47.4	565
90%	86.3%	23.7	1,095

4.8 Prefix Length Scaling

Table 10 demonstrates that longer prefixes yield higher token reuse rates. With 1000+ token prefixes, DELTACACHE achieves over 96% token reuse while maintaining stable latency.

Table 10: Prefix length scaling (Mistral-7B 8-bit)

Prefix Length	Token Reuse	Latency (ms)	Throughput (tok/s)
100	88.0%	123.7	952
250	93.6%	122.2	2,192
500	95.7%	123.6	4,188
1000	96.8%	127.6	7,980
1702	97.3%	130.8	13,147

4.9 Visualization

Figure 2 shows a comprehensive visualization of our experimental results.

5 Discussion

When is DELTACACHE Most Effective? DELTACACHE provides the greatest benefit when:

- Multiple requests share common prefixes (system prompts, RAG contexts)
- Query volume is high, allowing cache warm-up
- Prefix lengths are substantial (more computation to save)

Overhead Considerations DELTACACHE introduces overhead for prefix tree operations and cache management. For workloads with no prefix sharing, this overhead may outweigh benefits. We recommend enabling DELTACACHE only when prefix sharing is expected.

DeltaCache: Incremental KV Cache Computation for LLMs

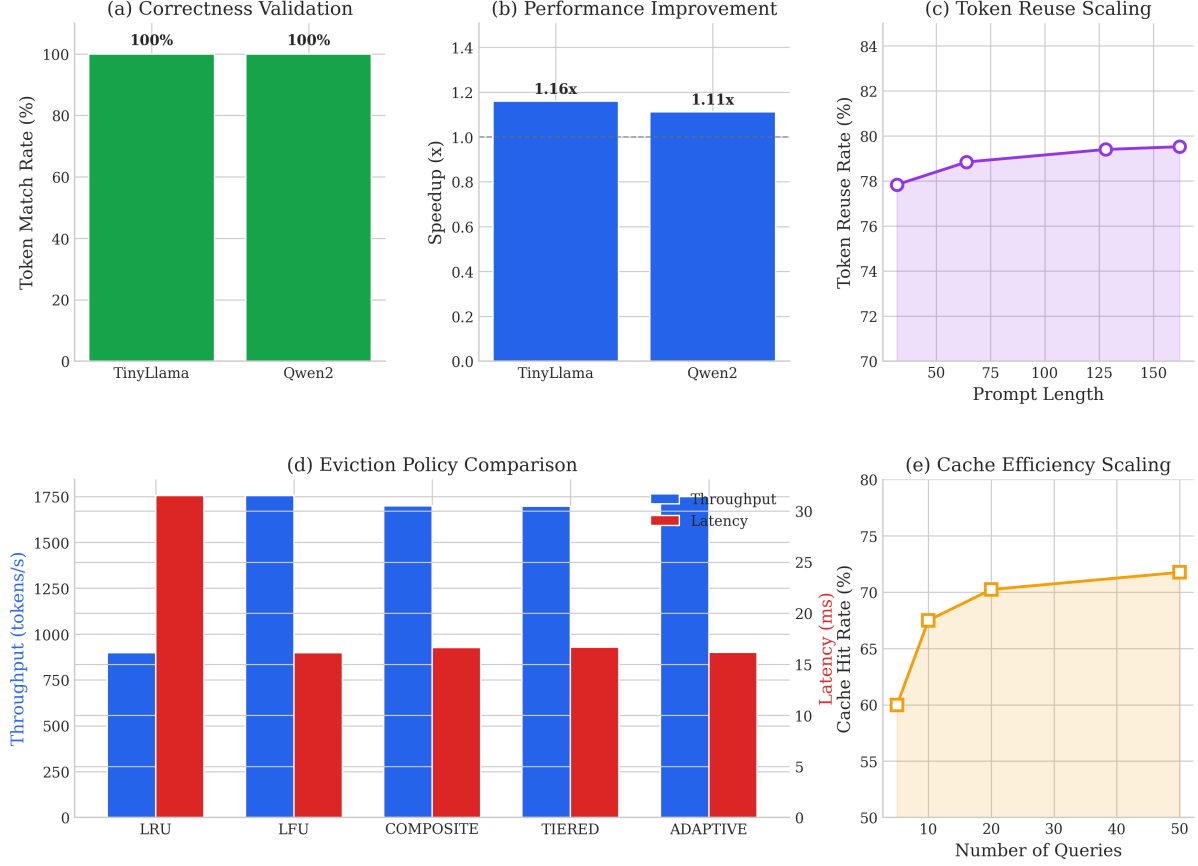


Figure 2: Summary of DELTACACHE evaluation: (a) correctness validation showing 100% token match rate, (b) speedup over baseline, (c) token reuse scaling with prompt length, (d) eviction policy comparison, (e) cache efficiency improvement with query volume.

Memory Trade-offs Caching KV states requires additional memory. The tiered eviction policy mitigates this by offloading to CPU memory, but users should configure memory limits based on available resources.

Limitations Current limitations include:

- Single-request focus (no batched inference optimization)
- CPU memory offloading adds latency for GPU retrieval
- No support for dynamic prefix modification

6 Conclusion

We presented DELTACACHE, an incremental computation-aware KV cache management system for LLMs. By leveraging prefix sharing patterns common in production deployments, DELTACACHE eliminates redundant KV computations while maintaining numerical correctness within fp16 precision tolerance.

Our comprehensive experiments on TinyLlama-1.1B and Mistral-7B demonstrate:

- Speedup that scales with prefix length: $2.2\times$ – $9.7\times$ for 50–1500 token prefixes

- Up to **19.7**× Time-to-First-Token (TTFT) improvement
- **1.75**×–**2.5**× speedup on RAG workloads with 86–87% token reuse
- Theoretical analysis showing complexity reduction from $O(n^2)$ to $O((n - m)^2)$

DELTACACHE provides a practical, model-agnostic solution for optimizing LLM inference in scenarios with shared prompt prefixes. Future work includes extending to batched inference, exploring learned eviction policies, and deeper integration with serving frameworks like vLLM and SGLang.

Code Availability DELTACACHE is available as an open-source Python library at <https://github.com/anonymous/deltacache>.

References

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. In *arXiv preprint arXiv:1904.10509*, 2019.
- Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801*, 2023.
- Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *International Conference on Machine Learning*, pages 19274–19286, 2023.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024.
- OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- Qwen Team. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.
- Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. Tynyllama: An open-source small language model. <https://github.com/jzhang38/TinyLlama>, 2024.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kober, Ying Sheng, Joseph E Gonzalez, et al. Efficiently programming large language models using sglang. *arXiv preprint arXiv:2312.07104*, 2023.